

Lectura 5

Aritmética de punto flotante

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Métodos y Cómputos Numéricos
Licenciatura en Ciencias de Datos
Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
26 ago 2025





Organización

INTRODUCCIÓN

ARITMÉTICA DE PUNTO FLOTANTE

EJEMPLOS

INTRODUCCIÓN

ARITMÉTICA DE PUNTO FLOTANTE

Un poco de historia

- Hace más de 40 años, cada computadora de uso personal tenía su sistema de número de punto flotante particular (decimales, binarias o triarias). Usaban base 2, 8 o 16 y todas tenían una precisión distinta.
- En 1985 el *Institute of Electrical and Electronics Engineers (IEEE)* y el *American National Standards Institute (ANSI)* adoptaron la **norma ANSI/IEEE Standard 754-1985** para la *Aritmética Binaria de Punto Flotante*.
- Resultado de una década de trabajo de cientos de matemáticos, científicos de computación, ingenieros, productores de computadoras, entre otros.
- Desde entonces las computadoras fueron diseñadas en la **Aritmética de Punto Flotante IEEE**.
- Esto no significa que todas lleguen al mismo resultado pero hay un modelo de cómo funciona la aritmética de punto flotante independiente de la máquina.

Representación en base β

Definición. Si $x \in \mathbb{R}$, la **representación en base β** es de la forma:

$$x \simeq \pm q \times \beta^n$$

donde β base, q mantisa, $\frac{1}{\beta} \leq |q| < 1$ y n exponente.

EJEMPLO: $\beta = 2$

$$x \simeq \pm q \times 2^n$$

donde $\frac{1}{2} \leq |q| < 1$.

Sin embargo en una computadora sólo tenemos una cantidad finita de números binarios para la mantisa q y el exponente n

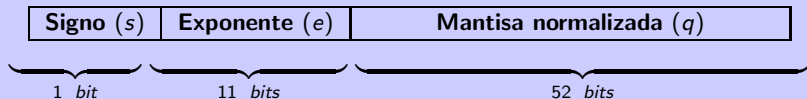
Ejemplo de punto flotante

Sean los reales positivos de la forma $0.d_1d_2d_3d_4 \times 2^n$ donde $d_1 = 1$, $d_i = 0, 1$ para $i = 2, 3, 4$ y $n \in \{-3, -2, -1, 0, 1, 2, 3, 4\}$.

Mantisa	Exponente							
	n=-3	n=-2	n=-1	n=0	n=1	n=2	n=3	n=4
$0.1000_{(2)}$	0.0625	0.125	0.25	0.5	1	2	4	8
$0.1001_{(2)}$	0.0703125	0.140625	0.28125	0.5625	1.125	2.25	4.5	9
$0.1010_{(2)}$	0.078125	0.15625	0.3125	0.625	1.25	2.5	5	10
$0.1011_{(2)}$	0.0859375	0.171875	0.34375	0.6875	1.375	2.75	5.5	11
$0.1100_{(2)}$	0.09375	0.1875	0.375	0.75	1.5	3	6	12
$0.1101_{(2)}$	0.1015625	0.203125	0.40625	0.8125	1.625	3.25	6.5	13
$0.1110_{(2)}$	0.109375	0.21875	0.4375	0.875	1.75	3.5	7	14
$0.1111_{(2)}$	0.1171875	0.234375	0.46875	0.9375	1.875	3.75	7.5	15

Punto flotante: IEEE754 doble precisión

- MATLAB/Octave/Scilab usa el formato de doble precisión de IEEE
- Cada número de doble precisión en aritmética de punto flotante se almacena en una palabra de 64 bits (8 bytes):



- La representación en número binario es:

$$s \mapsto d_1 \quad d_1 \in \{0, 1\}$$

$$e \mapsto d_1 d_2 \dots d_{11} \quad d_i \in \{0, 1\} \quad i = 1, \dots, 11$$

$$q \mapsto d_1 d_2 \dots d_{53} \quad d_1 = 1, \quad d_i \in \{0, 1\} \quad i = 2, \dots, 53$$

Punto flotante: IEEE754 doble precisión

Cada número en aritmética de punto flotante es normalizado:

$$x = (-1)^s (1 + (q)_2) 2^{e-1023} = (-1)^s (1.q)_2 2^{e-1023}$$

- El signo s surge de $(-1)^s$
- La mantisa (fracción) q en base 2 es un entero que satisface:

$$0 \leq (q)_2 < 1$$

o bien,

$$0 \leq (q)_{10} \leq 2^{52} - 1$$

En realidad, como $d_1 = 1$, el formato IEEE permite almacenar información de una palabra de 65 de bits, ¡en una de 64 bits!

Punto flotante: IEEE754 doble precisión

- Teniendo 11 bits para el exponente representarían:

$$0 \leq (e)_{10} \leq 2^{11} - 1 = 2047$$

- Como $(e)_{10} = 0$ y $(e)_{10} = 2047$ están reservados para números especiales, el rango es:

$$1 \leq (e)_{10} \leq 2^{11} - 2 = 2046$$

- Como $(e)_{10}$ no tiene signo, restando 1023 a la representación, se lo reacomoda para que sea un entero en el intervalo:

$$-1022 \leq (e')_{10} \leq 1023$$

Exponentes y números especiales

- Valores reservados del exponente:

$(e)_{10} = 0 \rightarrow$ todos ceros en el exponente

$(e)_{10} = 2047 \rightarrow$ todos unos en el exponente

(s)	$(e)_2$	$(q)_2$	Representa
0	todos ceros	todos ceros	0
1	todos ceros	todos ceros	-0
0	todos unos	todos ceros	Inf
1	todos unos	todos ceros	-Inf
0 o 1	todos unos	no-cero	NaN

- El mayor número positivo:

$$(1,11 \dots 1)_2 \times 2^{1023} \approx (1,7997)_{10} \times 10^{308}$$

- El menor número positivo:

$$(1,00 \dots 0)_2 \times 2^{-1022} \approx (2,2251)_{10} \times 10^{-308}$$

El *epsilon* máquina

- Una cantidad importante asociada a la aritmética de punto flotante es el **epsilon máquina** ε

El epsilon máquina es la distancia desde el número 1 al número de punto flotante siguiente más grande.

- Antes de la estandarización ANSI/IEEE había diferentes ε s para diferentes máquinas. Ahora en doble precisión IEEE

$$\varepsilon = 2^{-52}$$

- En MATLAB: $\text{eps} = 2.220446049250313\text{e-}16$
- En Octave: $\text{eps} = 2.22044604925031\text{e-}16$
- En Scilab: $\text{eps} = 2.22044604925\text{D-}16$

eps, realmin, realmax

Estos tres valores definen el rango de números disponibles y la precisión de nuestra computadora:

	Binario	Decimal
eps	2^{-52}	2.220446049250313e-16
realmin	2^{-1022}	2.225073858507201e-308
realmax	$(2-\text{eps}) \cdot 2^{1023}$	1.797693134862316e+308

- Cualquier número por encima de realmax genera **overflow**, y cualquier número debajo de realmin genera **underflow**.
- Algunas máquinas permiten **números de punto flotante subnormales** en el intervalo $[\text{eps} \cdot \text{realmin}, \text{realmin}]$. Esto permite llenar en parte el *gap* (salto) entre 0 y realmin.
- El menor número subnormal positivo de punto flotante es: 4.940656458412465e-324. Cualquier valor menor es cero.

Punto flotante: floatgui.m

- floatgui.m muestra la estructura de números de punto flotante
- El modelo del conjunto de los números positivos de punto flotante se determina por tres parámetros: t , $e_{\min}$ y $e_{\max}$
- Conjunto de números racionales de la forma $x = (1+q)2^e$ donde q :entero/ 2^t siendo $0 \leq q < 1$ y donde e :entero siendo $e_{\min} \leq e \leq e_{\max}$
- IEEE 754 doble precision tiene $t = 52$, $e_{\min} = -1022$, $e_{\max} = 1023$

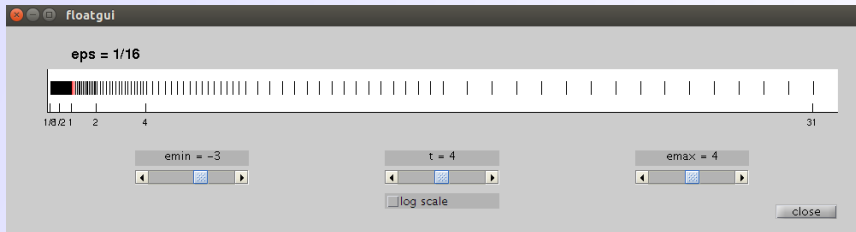


Figura: Estructura de números de punto flotante

EJEMPLOS

Errores en punto flotante

EJEMPLO 1:

El número decimal $4/3$ no es exactamente representable como una fracción binaria exacta. Por eso, el cálculo $1 - 3*(4/3 - 1)$ no resulta cero.

```
>> format long
>> a = 4/3
a = 1.3333333333333333
>> b = a - 1
b = 0.3333333333333333
>> c = 3*b
c = 1.0000000000000000
>> e = 1 - c
e = 2.220446049250313e-16
```


Errores en punto flotante

EJEMPLO 2:

El número decimal 0.1 no tiene representación exacta en el sistema binario debido a que su representación en binaria es periódica:

```
>> a = 0.0;  
>> for i = 1:10  
>>   a = a + 0.1;  
>> end  
>> a == 1  
ans = 0
```

Errores en punto flotante

EJEMPLO 3:

Notemos que el orden de las operaciones puede influir en el cálculo:

```
>> b = 1e-16 + 1 - 1e-16;  
>> c = 1e-16 - 1e-16 + 1;  
>> b == c  
ans = 0
```

EJEMPLO 4:

Notemos que hay un redondeo simétrico binario:

```
>> eps  
ans = 2.2204-16  
>> 1 + eps/2  
ans = 1  
>> 1+eps  
ans = 1.0000000000000000
```

Bibliografía

- **[KiCh1994]** D. Kincaid - W. Cheney, *“Análisis Numérico. Las matemáticas del cálculo científico”*, Addison-Wesley Iberoamericana, Delaware, 1994.
- **[MaFi2000]** J.M. Mathews - K.D. Fink, *“Métodos Numéricos con MATLAB”*, 3era Edición, Pearson Prentice Hall, Madrid, 2000.
- **[Mole2015]** C.B. Moler, *“Numerical computing with MATLAB”*, SIAM, Philadelphia, 2015.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *“Notas de Clases de Informática Aplicada y Métodos Computacionales”*, FCEIA, UNR, 2010.